Getting started

Part I Foundations

Scope and prerequisites

Stable insertion and merging with explicit boundaries establish loop and induction proofs. Track the hole, record preservation, inversions, and RAM costs, then derive merge-sort time and peak space for arbitrary lengths. Prerequisites are arrays, loop invariants, elementary recursion, and logarithms.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

- 2.1 Insertion sort. Prove correctness with loop invariants.
- 2.2 Analyzing algorithms. Derive worst-case bounds from line counts in the RAM model.
- 2.3 Designing algorithms. Formulate and solve the merge-sort recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.1 Insertion sort

Records have comparable keys and immutable identities. Keys form a total order; one comparison and one record move each cost a constant. Output keys must be nondecreasing, with each original record occurring exactly once. Stability additionally preserves equal-key order. Indices start at 1; empty intervals are never read.

An incremental algorithm maintains a sorted prefix and inserts the next record. In-place means constant extra words; it does not mean that every intermediate array is sorted or has no temporary duplicate values. Logical `and` short-circuits from left to right. The variable `key` stores a record; if records are object references, sorting moves references without changing the objects. Inputs of length at most 1 return immediately, so the initialization does not assume an existing first record for empty input.

Stable in-place insertion

```
INSERTION-SORT(A)
    n = A.length
    if n <= 1: return
    j = 2
    while j <= n
        key = A[j]
        i = j - 1
        while i >= 1 and A[i].key > key.key
            A[i+1] = A[i]
            i = i - 1
        A[i+1] = key
        j = j + 1
```

Shift only on strict greater-than, so equal keys never pass the new record. At $i = 0$, short-circuit evaluation prevents an out-of-bounds read. Each inner iteration decreases i, giving at most $j - 1$ iterations. Each outer iteration increases j, giving at most $n - 1$. Both loops terminate.

The outer invariant proves order and preservation

Let S be a copy of the initial input. Before each outer test, $A[1..j-1]$ is the **stable sorting** of $S[1..j-1]$, and $A[j..n] = S[j..n]$. Identities distinguish equal-key occurrences.

Initially $j = 2$, so the one-record prefix is sorted and stable, with unchanged suffix. The inner loop shifts exactly the suffix whose keys are strictly greater than key. Inserting key to its left preserves the old prefix's order and leaves every equal key before the new record. The extended prefix is the stable sorting of $S[1..j]$; the suffix is unchanged. At $j = n + 1$, the invariant gives the required stable sorting of the entire input. The early return separately handles $n \leq 1$. The invariant establishes the result; the iteration bounds establish that it is reached.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Preservation means array minus hole plus key

Fix an insertion j, and let B be the starting $A[1..j]$, whose first $j-1$ records are sorted. Treat $h = i + 1$ as a logical hole. Its old memory value is excluded from the representation.

Before each inner test,

$$A[1..i] = B[1..i], \qquad A[i+2..j] = B[i+1..j-1].$$

Every key in the right interval exceeds the saved $key = B[j]$. The nonhole positions together with key preserve exactly all records of B. Initially $i = j - 1$, and the right interval is empty.

If $A[i] > key$, copying $A[i]$ into the hole and decreasing i moves the hole left and preserves both equalities. At $i = 0$ or $A[i] \leq key$, all left keys are at most key, and all right keys exceed it. Filling the hole gives a sorted permutation. Claiming that the physical array remains a permutation after every shift is false; the logical hole excludes the duplicate created by copying.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Complete insertion trace

Input is [(7,a),(2,b),(5,c),(2,d),(1,e)], abbreviated 7a and so on. Identity is not a secondary sorting key.

j, saved record	Key comparisons from right to left	Shifts h_j	Array after filling hole
2, 2b	$7 > 2$ true; boundary	1	2b,7a,5c,2d,1e
3, 5c	$7 > 5$ true; $2 > 5$ false	1	2b,5c,7a,2d,1e
4, 2d	$7 > 2, 5 > 2$ true; $2 > 2$ false	2	2b,2d,5c,7a,1e
5, 1e	$7 > 1, 5 > 1, 2d > 1, 2b > 1$ true; boundary	4	1e,2b,2d,5c,7a

Record 2b always precedes 2d. There are 8 shifts, 10 key comparisons, and 12 complete inner-guard evaluations. Two guard evaluations stop at $i = 0$ without comparing keys. These counts are distinct.

Every state within one insertion

For $j = 4$, let $B = [2b, 5c, 7a, 2d]$ and save 2d. The square denotes a logical hole, not an extra array write.

i before test	Nonhole representation	Action
3	[2b,5c,7a,□] + 2d	$7 > 2$, write 7a to $A[4]$
2	[2b,5c,□,7a] + 2d	$5 > 2$, write 5c to $A[3]$
1	[2b,□,5c,7a] + 2d	$2 > 2$ false; stop
1	[2b,2d,5c,7a]	fill $A[2]$ with saved record

Every state within one insertion (continued)

One insertion, j=4: save key, shift, then fill the hole

i=3: move 7a into the right hole	2b	5c	7a	□	saved key=2d
i=2: move 5c into the right hole	2b	5c	□	7a	saved key=2d
i=1: 2b≤2d, stop	2b	□	5c	7a	saved key=2d
Fill with 2d; 2b stays first	2b	2d	5c	7a	Stable; all records preserved

□ marks an excluded logical slot. Its old memory value is not a second logical record.

Parameters, aliases, and control semantics

Arguments are passed by value. An array value is a reference to storage, so assigning $A[i]$ changes storage visible to the caller. `B=A` copies the reference and aliases the same storage; `B=copy(A)` creates an independent array. Rebinding local `A` to another array does not rebind the caller's variable.

For $A = [3, 1]$, after `B=A`, assigning $B[1] = 8$ also makes $A = [8, 1]$. With `B=copy(A)`, that assignment leaves A unchanged. Record identity differs from a memory address; sorting may rearrange references without reconstructing objects.

Indentation defines blocks; `return` ends the current call. NIL means no object, not a legal array position. The final failed while guard also costs time. A description such as “sort this set” cannot count as one constant step if it hides a variable-length computation.

2.2 Analyzing algorithms

A single-processor word-RAM executes loads, stores, comparisons, branches, and bounded-word arithmetic sequentially. Array access costs a constant. A word must represent indices, typically $w = \Theta(\lg(n+1))$ bits. Arbitrarily long integers cannot be placed in one word with every operation declared constant-time.

The model does not directly charge cache misses, page faults, external I/O, or parallel work. Implementations and hardware determine physical time. Comparing long strings can require reading many characters, changing the constant-comparison assumption. Arithmetic exceeding one word needs an explicit multiword representation and cost.

Input size depends on the problem. Sorting uses record count n, an integer problem may use bit length L, and graph problems commonly use both V, E. Numerical value N and encoded length $\lg N$ are not interchangeable. Counts below measure primitive operations, not seconds.

Count each line

For $n \geq 2$, let h_j be insertion j's shift count, $t_j = h_j + 1$ its inner-guard count, and $I = \sum h_j$. Initialization of $j = 2$ occurs once; the outer guard executes n times.

Operation	Count
Set n; test early return; initialize j	1 each
Outer guard	n
Save key; initialize i; fill hole; increment j	$n - 1$ each
Inner guard	$I + n - 1$
Shift; decrement i	I each

With fixed positive operation costs independent of n,

$$T(A) = c_0 + c_1 n + c_2(n-1) + c_3(I + n - 1) + c_4 I = \Theta(n + I).$$

Count each line (continued)

Here c_2 combines four operations and c_4 combines two. Including empty and singleton inputs gives $\Theta(1 + n + I)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Shifts count inversions exactly

An inversion of the original input S is a pair (u, v) with $u < v$ and $S[u].key > S[v].key$. Equal keys do not form inversions.

Before insertion j, the sorted prefix preserves the first $j - 1$ records. Those with keys greater than $S[j]$ form precisely one suffix. Each shifts once; no other record shifts. Therefore h_j counts exactly inversions with right endpoint j. Every inversion has one right endpoint, giving $\sum h_j = I(S)$ for all inputs.

The example has counts 1,1,2,4 and $I = 8$. Sorted input has $I = 0$ and time $\Theta(n)$. Distinct reverse-sorted keys have $I = n(n - 1)/2$ and time $\Theta(n^2)$. Few inversions yield nearly linear time, an adaptive property.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Comparisons, writes, best case, and worst case

Insertion j performs $h_j + 1$ key comparisons when $h_j < j - 1$, including a final false comparison; otherwise it performs h_j, stopping by short-circuit at zero. If b insertions reach the far left,

$$C_{key} = I + n - 1 - b, \quad C_{guard} = I + n - 1, \quad W_{array} = I + n - 1.$$

Saving `key` is not an array write. Sorted input has $h_j = b = 0$ and $C_{key} = n - 1$. Distinct reverse order has $h_j = j - 1$, $b = n - 1$, and $C_{key} = n(n-1)/2$. Equal lengths can have different costs.

Define $T_{best}(n) = \min_{|x|=n} T(x)$ and $T_{worst}(n) = \max_{|x|=n} T(x)$. Best and worst specify a function; O, Ω, and Θ specify bounds on that function. O does not mean worst case, nor Ω best case.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Average costs require a distribution

Assume all $n!$ permutations of **distinct** keys are equally likely. For $u < v$, let X_{uv} indicate $S[u] > S[v]$. Swapping the two positions is a bijection between either relative order, so $\Pr(X_{uv} = 1) = 1/2$. Linearity gives

$$I = \sum_{u<v} X_{uv}, \qquad E[I] = n(n-1)/4.$$

The indicators need not be independent; shared positions generally create dependence. Since $T(A) = \Theta(1 + n + I)$ with uniform constants across inputs, $E[T] = \Theta(n^2)$.

This is not a claim about every real dataset. A distribution concentrated on sorted inputs has expectation $\Theta(n)$. Random-input expectations and expectations over an algorithm's internal randomness on fixed input use different probability spaces.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

From exact costs to growth

With fixed RAM costs, worst-case time has form $an^2 + bn + c$, with $a > 0$. For sufficiently large n, $|b|n + |c| \leq (a/2)n^2$, yielding

$$(a/2)n^2 \leq T_{worst}(n) \leq (3a/2)n^2.$$

This proves the Θ bound. Constants and lower-order terms still affect finite inputs. In the same units, $1000n$ exceeds n^2 for $1 < n < 1000$, but is smaller for $n > 1000$. Ignoring finitely many sizes asymptotically does not excuse incorrect empty or singleton behavior. A constant-time early-return branch does not make every other input cheaper.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.3 Designing algorithms

Divide and conquer solves strictly smaller instances and combines their results. State boundaries, base cases, merge preconditions, and results. For $A[p..r]$, length is $m = r - p + 1$, with $p > r$ denoting an empty interval.

```
MERGE-SORT(A,p,r)
    if p >= r: return
    q = p + floor((r-p)/2)
    MERGE-SORT(A,p,q)
    MERGE-SORT(A,q+1,r)
    MERGE(A,p,q,r)
```

A legal nonempty call has $1 \leq p \leq r \leq n$. The full call is $(1, n)$, giving immediate return at $(1, 0)$ for empty input. Child lengths are $\lceil m/2 \rceil$ and $\lfloor m/2 \rfloor$, both smaller for $m \geq 2$. Computing the midpoint from a difference avoids the usual $p + r$ overflow, provided the index range itself is representable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

MERGE without an extra key value

Assume $1 \le p \le q < r \le n$ and two stably sorted halves. L and R are **independent copies**, not views affected by overwriting A. Their lengths are $q - p + 1$ and $r - q$.

```
MERGE(A,p,q,r)
    L = copy(A[p..q]); R = copy(A[q+1..r])
    i = 1; j = 1; k = p
    while i <= L.length and j <= R.length
        if L[i].key <= R[j].key
            A[k] = L[i]; i = i+1
        else
            A[k] = R[j]; j = j+1
        k = k+1
    while i <= L.length
        A[k] = L[i]; i = i+1; k = k+1
    while j <= R.length
        A[k] = R[j]; j = j+1; k = k+1
```

Choose L on equality. Every original left-half record precedes every right-half record, so stability is preserved. Length checks require no key-domain value named infinity.

Proof of merge preservation, order, and termination

Before the main loop,

$$k - p = (i - 1) + (j - 1).$$

The output $A[p..k-1]$ is exactly the stable merge of consumed prefixes $L[1..i-1], R[1..j-1]$. It is sorted, and every emitted key is at most every unconsumed key. Remaining records still exist in independent buffers.

The empty output establishes initialization. When both inputs remain nonempty, their heads are their smallest remaining records. Choosing the smaller head, with left priority on ties, gives the next record of the stable order. Consuming one record and incrementing k preserves lengths, data, order, and stability. Remaining record count decreases by one, proving termination.

At main-loop exit, at least one buffer is exhausted. The other is sorted with keys no smaller than the output tail, so appending it preserves the invariant. Finally $i = |L| + 1, j = |R| + 1, k = r + 1$, and all m records occur once. Without copies, an output write could destroy unread input and invalidate the proof.

Complete merge with duplicate keys

Let $L = [1a, 4b, 4c]$, $R = [2d, 4e, 6f]$, with L originally before R.

i,j before test	Comparison	Output prefix
1,1	$1a \leq 2d$	1a
2,1	$4b > 2d$	1a,2d
2,2	$4b \leq 4e$	1a,2d,4b
3,2	$4c \leq 4e$	1a,2d,4b,4c
4,2	L exhausted; no key comparison	append 4e,6f

There are 4 key comparisons, 6 output writes, and 6 copied buffer records. Interleaving until the last two records can require $m - 1 = 5$ comparisons. Θ(m) work does not mean exactly m key comparisons.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete merge with duplicate keys (continued)

Consume one, emit one; output writes cannot corrupt copied inputs

	L			R			
Start	1a	4b	4c	2d	4e	6f	Output: empty
After 1a,2d	—	4b	4c	—	4e	6f	Output: 1a,2d
Take left on ties	—	—	—	—	4e	6f	Output: 1a,2d,4b,4c

L exhausted: append 4e,6f. Equal-key records 4b,4c,4e retain their original order.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sentinel preconditions

A sentinel implementation can append an infinity record to each buffer and select the smaller head exactly m times, avoiding repeated exhaustion tests. The sentinel key must strictly exceed every legal key, and its record must not be legal input. The maximum legal integer is unsafe as a sentinel.

If legal keys include $+\infty$, take $L = [+\infty]$, $R = [+\infty, +\infty]$. Left priority can select the real left record, then its equal-valued sentinel, then read beyond L on the third output. This is not an out-of-domain marker.

Length checks work for any totally ordered key domain. A tagged sentinel is another option, but it must rank strictly above every real record and never be emitted as data.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

From merge to complete sorting

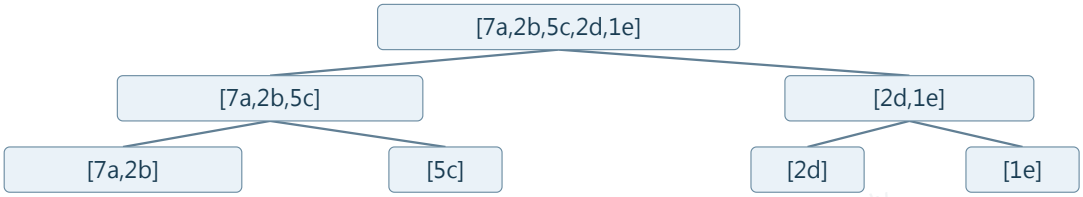
Use strong induction on interval length m. At $m \leq 1$, input is sorted and unchanged. For $m \geq 2$, both smaller calls stably sort and preserve their records by induction. MERGE's preconditions hold, and its proof gives a stable sorting of the whole interval, leaving outside storage unchanged. Child sizes decrease to 0 or 1, proving termination without a power-of-two assumption.

For [7a,2b,5c,2d,1e], splits have lengths 3 and 2, then 2/1 and 1/1. The figure shows every merged node. At the root tie, left 2b still precedes right 2d.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

From merge to complete sorting (continued)

Five records: split downward, return stable sorted results



Bottom up: [2b,7a] → [2b,5c,7a]; right half → [1e,2d]

Root merge: [1e,2b,2d,5c,7a]; key-2 tie: left 2b before right 2d

Child lengths 3 and 2; singletons return. The final split of [7a,2b] into singletons is omitted.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Include merging in the recurrence

Midpoint computation costs Θ(1). Copying and outputting m records costs Θ(m), giving

$$T(m) = T(\lceil m/2 \rceil) + T(\lfloor m/2 \rfloor) + \Theta(m), \quad T(0), T(1) = \Theta(1).$$

For $m = 2^h$, first solve the exact surrogate $U(1) = d$, $U(m) = 2U(m/2) + cm$, with positive c,d. Depth $\ell < h$ has 2^ℓ nodes costing $cm/2^\ell$ each, hence cm per level. There are h internal levels and m leaves of cost d, so $U(m) = cm \lg m + dm$.

Actual internal costs lie between fixed positive multiples of size, and leaves have constant lower and upper costs. Apply the expansion to both bounds to prove Θ(m log m), rather than assuming one exact c describes every instruction and base case.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Arbitrary lengths and a six-record cost table

At depth ℓ, each surviving interval has size at most $\lceil m/2^\ell \rceil$, so height is at most $\lceil \lg m \rceil$. Intervals at one depth are disjoint with total size at most m, proving O(m log m).

At depths $\ell < \lfloor \lg m \rfloor$, each interval has size at least $\lfloor m/2^\ell \rfloor \geq 2$; none has become a leaf. Total size is exactly m, giving Ω(m) at each of $\lfloor \lg m \rfloor$ levels. For $m \geq 2$, this is at least $(\lg m)/2$, proving the lower bound for arbitrary m.

Depth for m=6	Sizes	Total internal size
0	6	6
1	3,3	6
2	2,1,2,1	4
3	1,1,1,1; two earlier leaves	0

Buffer copies and output writes each total 16, not 6 at every depth. There are six base cases.

Peak space differs from cumulative allocation

MERGE allocates L/R only after both recursive calls return and releases them after merging. A length-m merge holds m buffer records. Child buffers do not coexist with parent buffers, giving Θ(n) peak auxiliary record space plus Θ(log n) stack space, hence Θ(n) overall for $n \geq 2$.

Cumulative allocation may be Θ(n log n), but it is not simultaneous demand. Retaining completed buffers changes the lifetime model and invalidates this peak bound. A reusable n-record scratch buffer is also possible with the same nonaliasing requirement.

Insertion sort needs only a saved record and constant index words. This merge sort guarantees Θ(n log n) time but is not in-place. Both are stable under their respective strict-shift and left-tie rules.

Sources

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 2.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study